EFE

Essaim Fluide Éphémère

Architecture complète — du moteur latent au swarm agentic

Document d'analyse technique détaillée couvrant l'intégralité de la plateforme EFE : le moteur d'inférence latente (M1-M12), les couches d'augmentation (M14-M33), la coordination méta-essaim (A1), le Mode Nano (M50), la couche d'audit transverse, et l'écosystème opérationnel complet.

Partie I — La vision et la promesse fondamentale

1. Le problème de marché qu'EFE résout

EFE existe parce que les agents d'intelligence artificielle déployés en production en 2026 souffrent simultanément de quatre maladies structurelles qui rendent leur usage inacceptable dans les secteurs régulés. La première est la latence séquentielle : un agent IA moderne qui doit traiter cent documents réglementaires va appeler le LLM cent fois en série, ce qui se traduit par des temps de réponse de cinq à neuf minutes pour des workflows pourtant simples conceptuellement. La seconde est le coût cumulatif des appels LLM : à raison de dix à cinquante centimes par appel multiplié par des centaines d'appels par workflow et des milliers de workflows par jour, la facture mensuelle d'une entreprise utilisatrice dépasse rapidement des centaines de milliers d'euros, voire des millions, sans corrélation directe avec la valeur produite.

La troisième maladie est plus subtile mais plus grave : les LLMs hallucinent sur les sous-tâches déterministes. Lorsqu'un agent IA est chargé de parser un JSON, d'extraire une date au format standard, de valider un schéma XML, ou d'identifier une entité clinique précise comme une molécule médicamenteuse, le LLM produit des erreurs dans cinq à quinze pour cent des cas. Ces erreurs sont inacceptables dans des contextes où chaque décision doit être audit-traceable : un essai clinique dont la phase est mal identifiée, un contrat d'assurance dont le montant est mal extrait, un dossier patient dont l'allergie est manquée : ces erreurs ont des conséquences réelles, légales et humaines.

La quatrième maladie est l'opacité de l'audit trail. Quand un agent IA prend une décision à partir de cent appels LLM internes, il n'existe aucune trace structurée de ces cent appels. Le raisonnement intermédiaire de l'agent est perdu dans un magma de chain-of-thought non typé, non versé en base de données, non vérifiable. Or les régulateurs — FINMA en Suisse, FDA aux États-Unis, ANSM en France, EMA en Europe, les obligations DORA pour le secteur financier, les exigences de l'AI Act européen, les standards GxP pharmaceutiques — tous exigent un audit trail par décision, conservable plusieurs années, cryptographiquement vérifiable, replayable. Les agents IA classiques ne peuvent simplement pas fournir cela.

EFE répond à ces quatre maladies par une architecture fondamentalement différente, construite couche par couche depuis 2024. Le cœur de la différentiation tient en une phrase : au lieu d'utiliser un seul LLM monolithique qui doit tout faire mal, EFE utilise un essaim de nano-agents spécialisés qui s'exécutent en parallèle, chacun avec un domaine de compétence bien défini, schema-validated, audité individuellement, et chacun produisant une trace forensique ré-jouable des années plus tard.

Pour l'utilisateur final, le gain est mesurable : une réduction typique de dix à quarante fois sur la latence des workflows complexes, une réduction de cinquante à soixante-dix pour cent sur le coût LLM, une élimination complète des hallucinations sur les sous-tâches déterministes, et un audit trail conforme par construction. Pour l'entreprise cliente, le gain est stratégique : elle peut déployer en production des workflows IA dans des contextes auparavant interdits par la réglementation, ouvrant des marchés qui étaient fermés aux agents IA classiques.

2. La taille du marché ciblé

Le marché mondial des dépenses LLM en API atteint approximativement quatre-vingts milliards de dollars en 2026 selon les estimations croisées d'analystes industriels. Une fraction substantielle de ce dépenses, qu'on peut estimer entre trente et quarante pour cent, est gaspillée sur des sous-tâches

déterministes pour lesquelles l'usage d'un LLM est une aberration économique et qualitée. Cela donne un marché adressable, ce qu'on appelle le TAM dans le jargon des fonds, d'environ vingt-cinq à trente milliards de dollars en 2026. Les projections industrielles pour 2028 portent ce TAM à soixante à quatre-vingts milliards de dollars, porté par l'explosion attendue des déploiements d'agents IA en production dans les entreprises Fortune 500.

EFE n'a pas besoin de capturer la totalité de ce marché pour devenir une entreprise considérable. La capture d'un seul pour cent du TAM 2028 représenterait deux cent cinquante à trois cents millions de dollars d'ARR potentiel. Pour mettre ce chiffre en perspective, c'est l'ordre de grandeur d'un mid-cap SaaS à valorisation entre deux et cinq milliards de dollars. Le positionnement stratégique d'EFE n'est pas de concurrencer Anthropic ou OpenAI sur leur terrain — les LLMs généralistes — mais d'occuper la couche d'infrastructure qui les rend utilisables en régulation. EFE ne remplace pas Claude ou GPT-4 : il les rend auditables et économiquement viables dans des contextes où ils ne pouvaient pas l'être.

3. La thèse architecturale en une phrase

La thèse centrale d'EFE peut être formulée ainsi :

> *« Les agents IA en production sont lents et chers parce qu'ils appellent un LLM pour chaque sous-tâche. EFE leur fournit un essaim de nano agents spécialisés et déterministes, exécutables en parallèle, avec audit-trail par appel. »*

Cette phrase contient quatre engagements techniques distincts qui structurent l'intégralité de la plateforme. Premier engagement : la spécialisation. Chaque nano-agent fait une seule chose, et la fait bien ; il a un schéma d'entrée et un schéma de sortie strictement validés. Deuxième engagement : le déterminisme. Lorsque c'est possible (mode A, pure Rust), le nano-agent est mathématiquement reproductible ; même entrée, même sortie, éternellement. Lorsque ce n'est pas possible parce qu'on a besoin d'un LLM (mode C), le nano-agent est au moins schema-bound : la sortie du LLM est validée contre un schéma JSON strict, et si elle ne valide pas, on retry ou on échoue, mais on ne propage jamais une donnée mal formée.

Troisième engagement : la parallélisation. Le dispatcher EFE peut exécuter des dizaines, des centaines, voire des milliers de nano-agents simultanément, respectant les dépendances déclarées entre eux sous forme de graphe acyclique dirigé (DAG). C'est cette parallélisation qui produit le gain de latence spectaculaire : là où un agent IA classique fait cent appels en série pendant neuf minutes, EFE fait cent appels en parallèle pendant dix secondes. Quatrième engagement : l'audit trail par appel. Chaque invocation d'un nano-agent produit une ligne dans une chaîne d'audit HMAC, lienée cryptographiquement à la précédente, conservable sept ans ou plus, replayable indefinitely. C'est cette traçabilité par décision qui rend EFE compatible avec les exigences régulatoires les plus strictes.

Partie II — Le moteur d'inférence latente (M1-M12)

4. L'IP différentielle : l'espace latent partagé

Avant même de parler de nano-agents ou de Mode Nano, il faut comprendre que le cœur historique d'EFE est un moteur d'inférence radicalement non-conventionnel. Plutôt qu'une chaîne d'appels LLM textuels, EFE matérialise une population d'agents éphémères comme un tenseur mathématique résidant en mémoire haute bande passante. Chaque agent dans la population est un vecteur de dimension latente D, situé quelque part dans un espace abstrait que les concepteurs appellent l'ELS, pour Epistemic Latent Space — l'Espace Latent Épistémique. La population entière est donc un objet mathématique de forme N par D, où N est le nombre d'agents actifs et D la dimension de l'espace.

Cette représentation a une conséquence profonde : au lieu de parler entre eux par messages textuels, les agents EFE « parlent » en partageant leurs positions dans cet espace latent. La communication est tensorielle, pas linguistique. Le calcul qui fait converger les agents vers un consensus est un opérateur de fusion qui prend en entrée le tenseur N par D à un instant t et produit le tenseur N par D à l'instant t+1. C'est exactement le module M5 de l'architecture EFE, qu'on appelle les Fusion Kernels.

La beauté mathématique de cette approche est qu'elle résout simultanément plusieurs problèmes que les architectures multi-agents textuelles ne savent pas résoudre. Premièrement, la convergence vers un consensus est rapide et mesurable : on dispose d'un critère numérique pour détecter qu'un essaim a convergé (module M6, le Consensus Detector), au lieu de devoir interpréter des textes pour deviner si plusieurs agents sont d'accord. Deuxièmement, la robustesse aux agents déviants est naturelle : un agent qui s'éloigne du consensus voit sa position être statistiquement filtrée par l'opérateur de fusion, sans qu'il faille un mécanisme de vote textuel. Troisièmement, l'audit devient possible parce que chaque round d'inférence produit un tenseur inspectable, hashable, conservable.

5. Les modules fondamentaux M1 à M12

L'architecture historique d'EFE est découpée en douze modules numérotés, chacun avec une responsabilité bien isolée. Le module M1, le Core Runtime, est l'orchestrateur principal qui dirige le cycle de vie d'une requête d'inférence de bout en bout. Il reçoit la requête, matérialise la population d'agents, lance les rounds d'inférence, détecte la convergence, extrait le verdict, et renvoie la réponse au client.

Le module M2, le Memory Pool, gère l'allocation de la mémoire haute bande passante nécessaire pour héberger les tenseurs de population. C'est un sujet technique non-trivial parce que les GPU modernes ont une bande passante mémoire limitée, et que la matérialisation puis la dissolution rapide de populations complètes à chaque requête impose une discipline d'allocation très stricte pour ne pas saturer le GPU. Le module M3, le Genesis Engine, est responsable de la création initiale des N agents pour chaque requête : il tire des positions initiales dans l'espace latent selon une distribution paramétrable, en respectant le budget mémoire fixé par M2.

Le module M4, l'ELS proprement dit, matérialise l'espace latent partagé. C'est là que les agents vivent. L'ELS est dimensionné lors de la création de chaque swarm, et peut être paramétré selon la complexité du problème à résoudre : une dimension de soixante-quatre suffit pour une classification

simple, alors qu'une dimension de mille vingt-quatre est nécessaire pour des tâches plus complexes. Le module M5, les Fusion Kernels, contient les opérateurs de fusion proprement dits, et c'est ce module qui déploie les kernels CUDA hot-path en Phase 3 pour accélérer l'inférence sur GPU. La fusion est, dans son essence, une opération de type swarm attention : chaque agent regarde les positions de tous les autres agents pondérées par leur similarité, et met à jour sa propre position en conséquence. La structure mathématique de cette opération ressemble à celle des mécanismes d'attention dans les transformers, mais appliquée à une population d'agents au lieu d'une séquence de tokens.

Le module M6 est le Consensus Detector. Il détermine, à chaque round d'inférence, si la population a convergé suffisamment pour qu'on puisse extraire un verdict. Les métriques utilisées incluent la variance intra-population, la distance moyenne au centroïde, et la stabilité de cette distance sur les derniers rounds. Lorsque ces métriques tombent en dessous de seuils configurés, le M6 déclenche la dissolution. Le module M7, la Dissolution, prend le tenseur convergé et en extrait un verdict. La méthode canonique est l'extraction du centroïde — le point moyen de la population — puis le décodage de ce centroïde en une réponse exploitable par le client.

Plusieurs stratégies de décodage cohabitent dans M7. La plus simple est le nearest_anchor decoder : on a pré-enregistré des points dans l'espace latent — les ancres — qui correspondent à des classes ou des concepts sémantiques, et le verdict est l'ancre la plus proche du centroïde. Cette approche est particulièrement adaptée à la classification, et c'est sur elle que Mode 1, le cas d'usage canonique d'EFE, est construit.

Le module M8 est le SDK Python et les templates ML. Il expose l'API EFE de façon pythonique pour les ingénieurs ML qui veulent intégrer EFE dans leur pipeline d'apprentissage, leur pipeline de pré-traitement de données, ou leur notebook Jupyter d'exploration. Le module M9, l'Alignment Registry, est le registre des cones d'ancres, c'est-à-dire des sets de points pré-enregistrés dans l'espace latent qui définissent les classes possibles d'un classifieur. Les cones sont versionnés, signés, et associés à un tenant pour la sécurité multi-tenant.

Le module M10 est la surface gRPC d'EFE. C'est ici que les clients externes se connectent. Le serveur tonic-based offre des méthodes pour soumettre des requêtes d'inférence, lister les cones disponibles, gérer les tenants, consulter les logs d'audit, et administrer les quotas. C'est aussi M10 qui expose le SDK Python généré automatiquement à partir des descripteurs proto, permettant à un développeur Python de parler à EFE comme à n'importe quelle bibliothèque asynchrone Python.

Le module M11 est la Télémétrie. C'est la couche d'observabilité qui produit les traces OpenTelemetry, les métriques Prometheus, et les logs JSON structurés pour chaque opération EFE. Chaque round d'inférence, chaque appel API, chaque décision d'admission est instrumentée. Cette instrumentation est ce qui permet aux équipes SRE de comprendre le comportement de la plateforme en production, de détecter les anomalies, et de respecter les SLO contractuels avec les clients. Le module M12, enfin, est le packaging : les Dockerfiles, les Helm charts, les manifestes Kubernetes, les scripts de build, tout ce qui permet de déployer EFE en production.

6. La distinction entre nano-agent et agent IA classique

Avant de poursuivre, il est utile de préciser pourquoi les concepteurs d'EFE ont choisi le mot « nano-agent », et en quoi un nano-agent diffère fondamentalement d'un agent IA tel qu'on l'entend habituellement quand on parle de Claude, GPT-4, ou Gemini. Un agent IA classique est multi-tour, stateful, probabiliste, multi-capability, lent, cher, et tend à halluciner sur les sous-tâches

déterministes. Le mot « agent » renvoie ici à un acteur intelligent qui possède une mémoire conversationnelle, peut prendre des décisions tactiques au cours d'une conversation, et est susceptible d'utiliser des tools pour interagir avec son environnement.

Un nano-agent, dans la terminologie EFE, est tout le contraire. Il est single-tour : il reçoit une entrée, produit une sortie, point. Pas de conversation. Il est stateless : aucune mémoire d'un appel précédent. Il est déterministe ou schema-bound : si c'est du pur Rust, il est strictement reproductible ; si c'est un appel LLM, sa sortie est validée contre un schéma JSON strict. Il est single-capability : il fait une seule chose. Il est rapide : dix à cinq cents millisecondes typiques. Il est bon marché : un à cent millicents par appel. Il est prédictible : ses comportements sont couverts par des tests automatisés, et sa sortie respecte toujours son schéma déclaré.

Cette distinction est fondamentale pour comprendre l'intégralité d'EFE. Les deux mots, « nano » et « agent », ne se réfèrent pas à la même chose dans les deux usages. Quand un client EFE parle d'un « agent IA » qui utilise EFE, il parle de Claude ou GPT. Quand le même client parle d'un « nano-agent », il parle d'une unité de calcul spécialisée fournie par EFE. Le schéma typique d'utilisation est : l'agent IA classique délègue ses sous-tâches déterministes à un essaim de nano-agents EFE, qui les exécutent en parallèle, beaucoup plus rapidement et plus fiablement que si l'agent IA classique avait tenté de les faire lui-même par appels LLM séquentiels.

Partie III — Les modes opérationnels d'EFE

7. Mode 1 : la classification par essaim

Mode 1 est le cas d'usage canonique d'EFE en production, celui qui a structuré la conception de toute la plateforme. Mode 1 transforme le moteur d'essaim vectoriel des modules M1 à M9 en un classifieur distribué robuste par consensus, appelable comme une fonction pure depuis n'importe quel agent LLM, sans que celui-ci ait conscience du mécanisme sous-jacent. C'est la propriété clé qui rend EFE adoptable : pour le développeur d'un agent IA, EFE est juste un tool de plus dans son palette, qu'il invoque quand il a besoin de classifier un texte. Toute la complexité de l'essaim — la matérialisation de la population, les rounds de fusion, la détection de consensus, la dissolution, le décodage — est cachée derrière cette interface simple.

L'agent IA, face à un texte libre de l'utilisateur, délègue la décision catégorielle à l'essaim, puis reprend son raisonnement une fois la classe et la confiance retournées. Concrètement, l'agent Claude qui reçoit la requête « Catalogue ce ticket support : 'Mon avion a été annulé et je veux un remboursement' » va, plutôt que tenter de classifier lui-même en interne, appeler le tool efe_classify avec le texte du ticket. EFE va alors démarrer un swarm de cent agents dans son espace latent, leur faire converger en quelques millisecondes vers un consensus, extraire le verdict, et le retourner : « refund_request, confiance 0.94, cone_version fin-v3, trace_id 8f3a-7b21-... ».

Mode 1 est composé de douze sous-systèmes documentaires, chacun avec sa propre architecture. Les ancres sémantiques sont le sous-système 1.1 : ce sont les points pré-enregistrés dans l'espace latent qui définissent les classes possibles. Le catalogue d'ancres est le 1.2 : c'est le registre versionné et tenant-scoped de ces ancres. Le cycle de vie des cones est 1.3 : comment on crée, met à jour, signe, promeut, et révoque les cones de production. Le bridge client Python est 1.4 : l'interface programmatique. Le choix de l'embedder est 1.5 : quel modèle ML utilise-t-on pour transformer le texte en vecteur initial dans l'espace latent. Le protocole gRPC utilisé est 1.6 : les messages Protocol Buffers exacts échangés entre le client et le serveur EFE. Le post-traitement du verdict est 1.7 : la mise en forme de la réponse, l'application d'éventuels filtres, l'enrichissement avec des métadonnées de traçabilité.

L'intégration agent LLM via tool est 1.8 : la conformité aux conventions Anthropic Tools, OpenAI Function Calling, LangChain Tools, et MCP. Le cache et idempotence est 1.9 : pour un même texte sur un même catalog_version, EFE garantit la même réponse, ce qui permet de mémoïser et d'optimiser le coût. L'observabilité côté client est 1.10 : les traces et métriques que le client peut consulter pour comprendre la latence et le coût. Les erreurs et fallback sont 1.11 : que faire quand EFE n'est pas disponible ou quand la confiance est trop basse pour être actionable. Et la sécurité multi-tenant est 1.12 : comment on garantit qu'un tenant ne voit pas les cones d'un autre.

Le budget de performances de Mode 1 est strict. En régime chaud, la réponse doit arriver en moins de deux secondes pour le 99-ième percentile, et en moins de cinq cents millisecondes pour le 50-ième percentile. Cette contrainte de latence guide toutes les décisions techniques : pas d'allocation mémoire dynamique pendant l'inférence, pas d'appel réseau externe, pas de I/O bloquant, pré-warming agressif du cache d'embeddings, pool de connexions gRPC pour les clients fréquents.

8. Mode 2 : les anchors LLM-routés

Mode 2, introduit avec le module M21, est une extension naturelle de Mode 1 qui répond à un problème concret : comment construit-on un cone d'ancres pour un nouveau domaine sans avoir une expertise ML pour le faire manuellement. Mode 2 utilise un LLM pour générer automatiquement les ancres à partir d'une description textuelle des classes à identifier. Le développeur fournit, par exemple, dix-huit phrases d'exemples pour chacune de ses douze classes ; un LLM génére ensuite des variations sémantiques de ces phrases, calcule leurs embeddings, et produit un cone d'ancres prêt à l'emploi.

Ce qui rend Mode 2 sophistiqué, c'est qu'il n'utilise pas un LLM monolithique fixé. Le module M21 définit un LLM router multi-provider : EFE peut router la génération d'ancres vers Anthropic, OpenAI, Ollama local, ou n'importe quel autre provider implementant l'interface standard. Le router applique des politiques de failover : si Anthropic est en panne, automatic failover vers OpenAI ; si OpenAI est aussi en panne, fallback vers Ollama. Le router applique aussi des politiques de budget : chaque tenant a une enveloppe LLM mensuelle, et le router refuse les appels qui dépasseraient cette enveloppe.

Le module M21 inclut également efe-prompts, qui est le registre versionné des prompts utilisés par tous les modes de génération de contenu d'EFE. Chaque prompt a un identifiant stable, une version semver, un template avec variables interpolées, et des métadonnées de provenance qui permettent à un auditeur de vérifier quel prompt exact a été utilisé pour générer un cone donné. Le registre supporte les expérimentations A/B : on peut déployer deux versions d'un prompt en parallèle, mesurer leurs performances respectives sur des datasets gold, et promouvoir la meilleure.

9. Mode 3 : le multi-turn iterative reasoning

Mode 3, introduit avec le module M22, ajoute le pattern de raisonnement multi-tour à EFE. Dans certains contextes, une seule classification ne suffit pas : il faut une cascade de classifications progressives, où la sortie d'une étape devient l'entrée de l'étape suivante. Mode 3 implemente cette cascade sous forme d'un concept stack : une pile de concepts progressivement raffinés.

Concrètement, supposons qu'on traite un ticket support. La première passe classifie le ticket comme « financial_complaint ». La deuxième passe, sachant maintenant qu'on est dans le domaine financier, classifie plus précisément comme « credit_card_dispute ». La troisième passe, sachant qu'on a une dispute carte de crédit, classifie le type d'action attendue : « chargeback_request ». À chaque étape, l'incertitude est réduite, et le contexte spécifique est augmenté.

Mode 3 ajoute aussi un workflow d'approbation humaine pour les décisions borderline. Quand une étape de la cascade produit une confiance en dessous d'un seuil configurable, ou quand elle aboutit à une classe marquée « sensible » dans le catalog, le système met la décision en queue de review humain. Un opérateur spécialisé consulte la décision via une interface dédiée, peut l'approuver telle quelle, la modifier, ou la rejeter avec un commentaire. Cette boucle homme-dans-la-boucle est cruciale pour les contextes à haute responsabilité comme les décisions d'octroi de crédit ou les triages médicaux.

Partie IV — La couche d'augmentation (M20-M28)

10. Le BYO-LLM Agent SDK (M20)

Le module M20 répond à une question commerciale critique : comment permettre à un client enterprise d'utiliser son propre LLM interne, potentiellement fine-tuné sur ses données propriétaires, plutôt qu'un LLM généraliste externe. C'est une exigence fréquente dans les grands groupes bancaires ou pharmaceutiques qui ont investi des dizaines de millions dans leur propre infrastructure LLM et ne veulent pas la voir court-circuitée par EFE. Le BYO-LLM — Bring Your Own LLM — SDK fournit l'interface nécessaire : le client implemente une interface Rust ou Python qui décrit comment appeler son LLM, et EFE peut ensuite router toutes les opérations LLM-augmented vers ce provider custom.

Le SDK BYO-LLM ne se contente pas de l'interface technique. Il inclut aussi une bibliothèque d'outils d'analyse qui aident l'utilisateur à comprendre le comportement de son LLM dans le contexte EFE : matrices de confusion comparées entre bundles, propositions d'exemples additionnels pour améliorer un classifier qui sous-performe, génération de tests synthétiques pour valider une classification avant de la déployer en production, drafts de runbooks opérationnels pour les SRE qui doivent gérer le LLM custom.

11. Le taxonomy curation (M23)

Le module M23 répond à un problème concret de tout déploiement de classifier en production : la taxonomie initiale est presque toujours imparfaite. Au bout de quelques semaines de production, on découvre que telle classe est trop large et confond deux concepts qui auraient dû être séparés, ou inversement que deux classes sont en fait le même concept et devraient être fusionnées. M23 est l'outillage qui rend cette évolution gracieuse, auditée, et réversible. Le système peut suggérer automatiquement des splits ou des merges en analysant la distribution des scores de confiance, identifier les exemples mal classifiés, et proposer des taxonomies révisées validées par un LLM avant promotion en production.

Chaque opération de curation produit une trace dans la chaîne d'audit HMAC. Lorsqu'un opérateur décide de splitter la classe X en X1 et X2, l'opération est traçable : qui l'a décidée, à quel moment, sur quels exemples motivants, avec quelle justification textuelle, vers quelle nouvelle taxonomie. Cette traçabilité permet, des années plus tard, à un régulateur d'auditer non seulement les classifications individuelles, mais aussi l'évolution de la taxonomie elle-même.

12. L'active learning closed-loop (M24)

Le module M24 ajoute une dimension fondamentale : l'apprentissage continu. L'idée est simple en principe et raffinée en pratique. Le système identifie automatiquement les classifications dont la confiance est intermédiaire — ni très élevée, ni très basse — et les soumet à un LLM oracle qui produit une étiquette de référence. Ces étiquettes alimentent un dataset gold qui grossit continuellement. Périodiquement, le système re-tune les centroïdes des ancres sur ce dataset gold élargi, évalue les performances du nouveau cone sur un test set bloqué, et si les métriques s'améliorent significativement, déclenche une promotion canary vers la production avec rollback automatique sur détérioration.

La chaîne complète est : production produit des classifications avec scores de confiance, uncertainty sampling identifie les cas borderline, LLM oracle produit des étiquettes pour ces cas, gold dataset s'enrichit, centroid retune ajuste les positions des ancres, eval bloqué valide la qualité, canary deploy expose le nouveau cone à un sous-ensemble du trafic, métriques surveillées pendant une fenêtre, et finalement promotion ou rollback automatique. Tout ce pipeline produit des traces d'audit qui montrent, des années plus tard, comment chaque version du cone a émergé.

13. L'agent runtime sandbox : M25 et M26

Les modules M25 et M26 adressent un risque spécifique aux déploiements d'agents IA autonomes : l'agent peut, par mauvaise programmation ou attaque adverse, tenter d'exécuter des opérations interdites — exfiltrer des données, escalader des privilèges, contacter des serveurs externes non whitelistés. M25 isole chaque exécution d'agent dans une microVM Firecracker sandbox, avec une whitelist stricte des tools disponibles, un step approval obligatoire pour les opérations sensibles, et un DPIA — Data Protection Impact Assessment — automatique par action.

Firecracker est la technologie d'isolation utilisée par AWS Lambda : ce sont des microVMs ultra-légères qui démarrent en quelques centaines de millisecondes et offrent une isolation kernel comparable à des VMs traditionnelles, mais à un coût en latence et en mémoire un ordre de grandeur plus faible. C'est exactement ce dont EFE a besoin pour faire tourner des agents IA potentiellement non-fiables sans risquer de compromettre l'infrastructure hôte. Le module M26 est la graduation production de M25 : passage de l'état EXPERIMENTAL au statut production-ready avec SLA contractuels.

Chaque pas d'exécution d'un agent dans le sandbox produit un trace forensique : quel tool a été appelé, avec quels arguments, par quel agent, dans quel contexte, à quel moment. Quand un tool sensible est sur le point d'être invoqué — par exemple, l'envoi d'un email externe, l'écriture dans une base de données de production, l'appel d'une API financière externe — le système déclenche un step approval : l'agent est mis en pause, l'opérateur humain reçoit une notification, et l'exécution ne reprend qu'après approbation explicite. Ce pattern est conforme aux exigences de l'Article 9 de l'AI Act européen pour les systèmes IA à haut risque.

14. Le cache plane Redis (M27)

Le module M27 ajoute à EFE une couche de cache distribuée basée sur Redis Cluster. Cette couche sert trois usages distincts mais coordonnés. Le premier est le cache d'embeddings : pour un même texte sur une même version d'embedder, l'embedding calculé est caché pour éviter de le recalculer à chaque requête. Cela est particulièrement bénéfique pour les workloads où les mêmes textes reviennent souvent — classification de templates d'email, de tickets support standardisés, ou de documents récurérents.

Le deuxième usage est le cache des réponses LLM. Quand un appel LLM est fait avec un prompt strictement identique à un appel précédent, on retourne la réponse cachée. Cela réduit drastiquement le coût LLM — on observe typiquement des ratios de cache hit de soixante à quatre-vingts pour cent en production avec workloads enterprise — et améliore la latence puisqu'on évite l'aller-retour réseau vers le provider LLM. Le troisième usage est les compteurs de quota par tenant : combien d'appels par minute, combien d'appels par jour, combien de dollars dépensés par mois. Ces compteurs sont consultés à chaque appel pour faire respecter les contrats commerciaux.

15. Les events NATS et webhooks (M28)

Le module M28 ajoute une couche événementielle à EFE basée sur NATS JetStream. C'est la couche qui permet d'intégrer EFE avec les systèmes downstream — CRM, ticketing, ERP, datalakes, systèmes de monitoring — via des événements typés et durables. Chaque opération significative dans EFE peut émettre un événement : une nouvelle classification effectuée, un drift détecté sur un cone, une promotion de bundle, une suspension de tenant, un budget LLM épuisé. Ces événements sont publiés dans des streams NATS JetStream, garantissant leur persistance et leur ordering.

Les consommateurs externes peuvent s'abonner à ces streams pour recevoir les événements et déclencher leurs propres actions. EFE supporte aussi le modèle webhook classique : un client peut enregistrer une URL HTTPS, et EFE va envoyer un POST signé cryptographiquement à cette URL pour chaque événement qui correspond à ses filtres. La signature cryptographique garantit l'authenticité : le destinataire peut vérifier que l'événement provient bien d'EFE et n'a pas été falsifié.

Partie V — La couche données (M29-M33)

16. La PII detection et auto-redaction (M29)

Le module M29 adresse l'exigence absolue dans les contextes régulés : aucune donnée personnelle identifiable, aucune information de santé protégée, aucune donnée financière client ne doit transiter vers un LLM externe sans avoir été préalablement redactée. Le module fournit trois couches collaboratives. La première est la détection : un système hybride régex plus modèle ML qui identifie les noms propres, adresses physiques, numéros de téléphone, numéros de sécurité sociale, numéros de carte de crédit, dates de naissance, diagnostics médicaux, molécules médicamenteuses prescrites à un patient individuel, et toute autre PII reconnue par les standards GDPR et HIPAA.

La deuxième couche est la canonisation et le redactor : chaque token PII détecté est remplacé par un token de redaction stable — typiquement de la forme [PII_PERSON_1], [PII_ADDRESS_2], etc. — qui préserve la structure sémantique du texte sans révéler son contenu personnel. La stabilité des tokens est cruciale : si la même personne est référencée plusieurs fois dans le même document, elle reçoit le même token chaque fois, ce qui préserve la capacité du LLM à raisonner sur les co-références sans connaître l'identité réelle. La troisième couche est le circuit breaker : si trop de PII sont détectées dans un même document, ou si le système détecte un échec de redaction (un token PII qui aurait fui dans la sortie), le circuit breaker se déclenche, bloque le traitement, et émet une alerte de sécurité.

17. Le multimodal : vision, audio, et long contexte (M30)

Le module M30 élargit l'horizon d'EFE au-delà du texte. Trois sous-modules coexistent. Le premier est la vision par SigLIP, un modèle contrastif image-texte qui produit des embeddings pour les images dans le même espace latent que celui utilisé pour le texte. Cela permet de classifier des images contre des cones d'ancres textuelles — par exemple, classifier des photos de produits selon leur catégorie marketing — en utilisant exactement la même machinerie d'essaim qui sert pour le texte. Le deuxième sous-module est l'audio par Whisper, le modèle de transcription speech-to-text d'OpenAI qui est aussi devenu un standard open-source. Whisper transforme un fichier audio en transcription textuelle qui peut alors être classifiée par EFE.

Le troisième sous-module est le long-context chunker. Quand un document est trop long pour tenir dans la fenêtre de contexte d'un LLM — typiquement plus de cent mille tokens — il faut le découper intelligemment en chunks plus petits, traiter chaque chunk séparément, puis fusionner les résultats. Le chunker EFE respecte les frontières sémantiques (paragraphes, sections, chapitres) pour éviter de couper en plein milieu d'une idée, et applique une stratégie de late fusion pour combiner les outputs des chunks en une réponse unifiée. L'ensemble est intégré avec un client Triton HTTP pour servir les modèles ML lourds depuis une infrastructure dédiée à haute disponibilité.

18. La region awareness et le DR (M30)

Le module M30 contient aussi efe-region, qui adresse les exigences de souveraineté des données et de continuité d'activité. Les clients européens ne veulent pas que leurs données transitent par des serveurs américains ; les clients gouvernementaux exigent une isolation géographique stricte ; les secteurs régulés doivent pouvoir prouver qu'en cas de catastrophe régionale — incendie de data center, panne électrique de quartier, attaque cyber massive — leur service reste accessible. Efe-region fournit la machinerie pour satisfaire ces exigences.

Concrètement, le module supporte des déploiements multi-régions où chaque tenant peut être lissé à une région spécifique — eu-west pour les clients européens, us-east pour les américains, ap-southeast pour les asiatiques — avec garantie que les données de ce tenant ne quittent jamais la région choisie. Le module surveille la santé de chaque région via des healthchecks croisés automatisés. En cas de défaillance détectée, un opérateur peut déclencher un failover manuel vers une région secondaire, qui prend en charge les requêtes le temps que la région principale soit réparée. Enfin, le module impose des drills DR trimestriels : tous les trois mois, les SRE doivent simuler une perte de région complète, exécuter le runbook de failover, et vérifier que le temps de basculement reste dans le budget contractuel.

19. Le metering et le billing Stripe (M31)

Le module M31 est composé de deux sous-modules : efe-metering et efe-billing. Le métering est la couche qui compte les usages en temps réel : chaque tenant, chaque opération, chaque appel LLM, chaque round d'inférence est compté. Les compteurs sont stockés dans Redis pour la latence sub-milliseconde, et périodiquement aggregés vers Postgres pour le reporting. La granularité est configurable : par minute pour les compteurs rolling window, par jour pour le reporting standard, par mois pour la facturation.

Le billing Stripe est la couche qui transforme les compteurs de métering en factures réelles. Le module supporte plusieurs plans tarifaires — Free, Developer, Pro, Enterprise, Audit-grade — chacun avec ses limites de volume, ses prix unitaires, et ses fonctionnalités incluses. La génération des factures mensuelles est automatisée : un job nocturne aggregue les usages du mois écoulé pour chaque tenant, calcule la facture selon son plan, et la pousse vers Stripe qui s'occupe de la collecte. Les budgets sont enforced à deux niveaux : un soft warning à quatre-vingts pour cent du budget mensuel, et un hard cap à cent pour cent qui déclenche un throttling automatique des requêtes.

20. Le vector index et le hybrid retrieval (M32)

Le module M32 ajoute à EFE une couche complète de recherche vectorielle comparable, en surface, à ce qu'offrent Pinecone, Weaviate, ou Qdrant en SaaS. Mais là où ces solutions sont des bases vectorielles externes appelables via API, M32 est intégré dans EFE avec garanties audit-grade. Le sous-module efe-vector fournit l'index vectoriel proprement dit : distance metrics (cosine, euclidean, dot product), index plat pour les petites collections, index HNSW pour les grandes, quantization pour réduire l'empreinte mémoire, persistance des collections sur disque ou objet store.

Le sous-module efe-retrieval ajoute par-dessus la couche d'orchestration hybride. La pratique moderne en recherche d'information est de combiner la recherche dense par vecteurs — qui capture la sémantique — avec la recherche sparse par BM25 — qui capture les matchs lexicaux exacts. Une requête utilisateur est exécutée en parallèle sur les deux index, et les résultats sont fusionnés par un algorithme appelé RRF, Reciprocal Rank Fusion, qui donne typiquement de meilleurs résultats que l'un ou l'autre seul. Le tout est exposé via une API consistante avec le reste d'EFE : tenant-scoped, audité, versionné.

21. Le knowledge graph et GraphRAG (M33)

Le module M33 est le plus récent et le plus ambitieux des modules augmentation. Il ajoute à EFE une couche de knowledge graph et l'orchestration GraphRAG qui en découle. Le sous-module efe-kg fournit le knowledge graph propre : entités typées (personnes, organisations, médicaments, concepts

légaux, instruments financiers), relations typées (X est un sous-type de Y, A interagit avec B, R est applicable à C dans le contexte D), et un moteur de traversal pour parcourir le graphe selon des patterns de requête.

Le sous-module efe-graphrag combine ce knowledge graph avec la recherche vectorielle de M32 dans un pattern appelé multi-hop GraphRAG. L'idée est de répondre à des questions complexes en plusieurs hops sur le graphe, chaque hop étant décidé par un LLM qui analyse les résultats partiels obtenus jusque-là. Une question typique serait : « Quels sont les médicaments contre-indiqués avec un anticoagulant chez un patient diabétique de plus de soixante-dix ans ? » Le système identifie l'anticoagulant dans le graphe, traverse les relations « contre-indiqué avec » vers les médicaments concernés, filtre par applicabilité au diabète, puis par seuils d'âge, et retourne la liste enrichie avec les références sources. Tout le pipeline est audité, replayable, et expose ses étapes intermédiaires pour la vérification par un expert médical.

Partie VI — Le Meta-Swarm (A1)

22. Le concept d'essaim d'essaims

Le Meta-Swarm, identifié dans la documentation par le code A1, répond à un problème spécifique qui émerge quand on opère plusieurs instances EFE indépendantes — par exemple, une instance par région géographique, ou une instance par département métier dans une grande entreprise — et qu'on souhaite que ces instances puissent collaborer sur des décisions transverses sans pour autant fusionner leurs états internes. Le Meta-Swarm est, conceptuellement, un essaim d'essaims : chaque participant est lui-même un swarm EFE complet, mais leurs résultats convergent dans un espace latent commun d'un niveau d'abstraction supérieur.

La mathématique sous-jacente est élégante. Chaque swarm participant produit, à chaque round de son inférence interne, un état tensoriel dans son propre espace latent. Cet état est ensuite projeté vers un espace latent méta, partagé entre tous les participants, par un opérateur appelé Lift. Dans l'espace méta, un opérateur MetaOperator combine les projections de tous les participants pour produire un état méta unifié. Cet état méta unifié est ensuite reprojeté vers chaque espace participant par un opérateur Pull, et injecté dans la prochaine itération de chaque swarm participant comme un terme de couplage.

Cette structure Lift-MetaOperator-Pull produit une dynamique de convergence couplée : chaque swarm continue d'évoluer selon sa propre dynamique interne, mais reçoit aussi un signal méta qui le tire vers le consensus collectif. L'opérateur Lift est typiquement une projection linéaire qui réduit la dimension de l'espace participant vers la dimension méta ; l'opérateur Pull est sa transposée qui ramene le signal méta dans la dimension du participant. Le MetaOperator peut être aussi simple qu'une moyenne pondérée des projections, ou aussi sophistiqué qu'un mécanisme d'attention sur les projections.

23. Les trois phases de déploiement A1

L'équipe EFE a structuré le déploiement du Meta-Swarm en trois phases délibeérement progressives, chacune avec son propre niveau de garantie et de complexité opérationnelle. La Phase 1, déjà livrée, fournit la primitive Meta-Swarm comme une bibliothèque Rust pure, in-process. Le crate efe-meta-swarm est compilable et testable en isolation, sans aucune dépendance vers le reste du workspace EFE. La Phase 1 inclut tous les blocs mathématiques : les espaces tensoriels et leurs opérateurs Hilbert, le trait SwarmParticipant qui décrit ce qu'est un participant abstrait, les opérateurs Lift Pull et MetaOperator avec leurs implementations canoniques, les ensembles de sécurité admissibles et les opérateurs de projection associés, la boucle d'itération couplée proprement dite, et le builder API fluent pour configurer un Meta-Swarm.

Le point clé de la Phase 1 est qu'elle ne modifie strictement rien dans le reste d'EFE. Le binaire de production efe-server reste byte-identique avec ou sans Phase 1. Cette propriété de non-régression stricte est appelée le « master-gate hook » dans la documentation, et elle est vérifiée par comparaison de hash SHA-256 sur le binaire compilé. C'est une discipline extrêmement stricte mais nécessaire : les clients enterprise qui ont signé des contrats avec garanties de stabilité ne veulent pas que l'ajout d'une fonctionnalité expérimentale puisse affecter leur production existante.

La Phase 2, également livrée, élève le Meta-Swarm d'une bibliothèque in-process à un coordinateur multi-process gRPC. Cela permet de déployer plusieurs participants dans plusieurs

processus, potentiellement sur plusieurs machines, communiquant via un mesh gRPC mTLS. Deux nouveaux services gRPC sont définis : MetaSwarmService, qui orchestre le coordinateur central, et MetaParticipantService, qui expose chaque participant individuel. Ces deux services sont déclarés comme additifs aux services efe.v1 existants : aucun breaking change, aucune modification de l'API existante.

La Phase 2 introduit un nouveau crate, efe-meta-swarm-server, qui expose l'API Phase 1 derrière un wrapper gRPC Tonic. Le binaire est packageable en Docker et déployable en Helm avec un sub-chart dédié. L'observabilité est intégrée via Prometheus et OpenTelemetry, avec des métriques spécifiques au Meta-Swarm : taux de convergence par participant, latence des opérations Lift et Pull, santé des connexions mTLS entre participants. Le configuration management supporte les variables d'environnement et le SIGHUP hot-reload pour les paramètres non-critiques.

La Phase 3, en cours de déploiement avec les PR 25 et 26, ajoute le hardening opérationnel, le support cross-tenant, et le scale-out. C'est la phase qui transforme le Meta-Swarm d'un système utilisable en production interne à un système déployable chez des clients enterprise avec SLA contractuels. Les éléments clés de la Phase 3 sont : le partitioning par tenant pour garantir l'isolation, le scale-out horizontal qui permet d'ajouter des coordinateurs à la volée sans interruption, les chaos drills pour valider la résilience face aux pannes réseau et aux participants malicieux, et l'instrumentation extensive pour l'analyse post-mortem.

24. La stratégie d'activation par niveaux

Une fois le Meta-Swarm techniquement livrable, se pose la question commerciale : comment l'activer chez les clients sans perturber les déploiements existants. L'approche EFE est une activation par niveaux successifs. Au niveau cluster, l'opérateur de l'instance EFE peut activer ou désactiver le Meta-Swarm pour tout le cluster via une variable d'environnement EFE_META_SWARM_ENABLED. Cette activation n'a aucun effet sur les workloads existants qui ne demandent pas explicitement le Meta-Swarm. Au niveau tenant, chaque tenant a un feature flag dans son enregistrement de base de données : tenant.features_jsonb avec une clé meta_swarm. Cela permet de déployer le Meta-Swarm en production pour des tenants pilotes avant le général release.

Au niveau request, chaque requête individuelle peut demander explicitement l'usage du Meta-Swarm via un paramètre meta_swarm_enabled dans le payload. Cela permet aux clients en exploration de mesurer le bénéfice du Meta-Swarm sur des workloads spécifiques sans engager l'intégralité de leur trafic. Au niveau session, enfin, le SDK supporte un mode persistent où toutes les requêtes d'une session client utilisent le Meta-Swarm. Ces quatre niveaux d'activation, cluster-tenant-request-session, sont implémentés indépendamment et combinables : une requête utilise le Meta-Swarm si et seulement si tous les quatre niveaux l'autorisent.

25. Les vingt-six pull requests qui composent A1 Phase 2

La Phase 2 du Meta-Swarm n'est pas un livrable monolithique : elle est composée de vingt-six pull requests numérotées PR5 à PR26, chacune avec son architecture doc détaillée et son scope strictement délimité. Cette discipline de découpage est typique de l'approche EFE : chaque PR doit pouvoir être reviewée indépendamment, doit pouvoir être rollbackée indépendamment, et doit produire un livrable testable en isolation. Les PR couvrent des aspects comme le scaffolding initial des proto et des binaires gRPC, l'implémentation progressive de la logique métier, l'observabilité, la sécurité, les benchmarks, les chaos tests, le load testing, la documentation, les SLO, les smoke checks, et finalement les PR 25 et 26 qui ferment la Phase 2 avec audit et leçons apprises.

Plusieurs PR émergent comme particulièrement structurantes. La PR 20 livre le loadgen L2 meta-swarm, un générateur de charge production-grade qui permet de mesurer le comportement du Meta-Swarm sous trafic réaliste. La PR 21 livre la bench harness, une plateforme reproductible de benchmark cross-version qui permet de comparer les performances entre releases successifs. La PR 22 livre le slos rollup, un pipeline de calcul des Service Level Objectives sur quatre tiers avec rollup multi-niveaux. La PR 23 livre la smoke runner, un système de neuf smoke checks exécutés à chaque merge sur la branche principale comme gate de qualité.

Partie VII — Le Mode Nano (M50)

26. La thèse Business-to-Agent

Le Mode Nano, identifié par le code M50, est la 4ème couche d'EFE et la plus récente. Sa thèse commerciale est d'aller au-delà du B2B traditionnel pour occuper un nouveau segment : le B2A, Business-to-Agent. L'idée fondatrice est que les agents IA des clients — Claude, GPT-4, Gemini Ultra, agents custom — deviennent eux-mêmes des clients d'EFE, achetant à l'usage des invocations de nano-agents pour accélérer leurs sous-tâches déterministes. Cela étend drastiquement le marché adressable d'EFE : au lieu de vendre une plateforme à des grandes entreprises régulées (le marché B2B), on vend des microservices fonctionnels à chaque agent IA en production dans le monde (le marché B2A).

Le positionnement stratégique du Mode Nano s'inscrit pile dans le mouvement actuel de l'industrie. Anthropic et OpenAI investissent massivement dans les capacités tool-use de leurs LLMs : Claude depuis la version 3.5, GPT depuis le function calling de 2023. Cette évolution rend les LLMs capables d'invoquer des services externes pendant leur raisonnement. Le Mode Nano se positionne comme le fournisseur canonique de tels services pour les sous-tâches déterministes : parse JSON, valid XML, extract regex, normalize dates, recognize entities, classify topics, summarize text. EFE est en cours de devenir à ces sous-tâches ce qu'AWS Lambda est aux fonctions serverless : l'option par défaut.

27. Les trois modes d'exécution

Le Mode Nano définit trois modes d'exécution distincts pour ses nano-agents, chacun adapté à une classe de tâches. Le premier est le Mode A, pure Rust. Un nano-agent en Mode A est une fonction Rust pure, exécutée dans un tokio thread pool sans aucune isolation Firecracker. Sa latence est de l'ordre du milliseconde, son coût est très faible, et son déterminisme est total : même entrée, même sortie, éternellement. Le Mode A est approprié pour les tâches purement computationnelles : parser un JSON contre un schéma, valider un XML, extraire un regex, normaliser une date, parser un SQL safely, et ainsi de suite. Dix nano-agents Mode A sont fournis dans le starter pack : json_validate, xml_validate, regex_extract, date_normalize, currency_normalize, markdown_extract_headings, csv_normalize, sql_parse_safe, html_extract_main_content, et pdf_extract_text.

Le deuxième mode est le Mode B, Local Model. Un nano-agent en Mode B utilise un petit modèle ML pré-chargé en mémoire : typiquement un modèle NER pour la reconnaissance d'entités, un classifier multilingue, un modèle de sentiment, ou un embedder. L'exécution se fait dans une microVM Firecracker isolée pour des raisons de sécurité et de prédictabilité de la latence. La latence typique est de cinquante à deux cents millisecondes, le coût est de l'ordre d'un millicent par appel. Cinq nano-agents Mode B sont fournis : ner_clinical pour la reconnaissance d'entités médicales, ner_legal pour le légal, intent_classify_generic pour la classification d'intentions, lang_detect pour la détection de langue, et embed_text pour la production d'embeddings généralistes.

Le troisième mode est le Mode C, LLM-augmented. C'est le mode le plus sophistiqué : un nano-agent en Mode C effectue un appel LLM via le router M21, avec strict JSON schema enforcement sur la sortie. Cela combine la flexibilité et la qualité d'un LLM avec la prédictabilité et la machine-readability d'un schema strict. Si le LLM retourne une sortie qui ne valide pas contre le schema, le nano-agent peut soit retry avec un prompt légèrement différent, soit échouer explicitement, mais ne propage jamais une donnée mal formée en aval. La latence typique est de

cinq cents à deux mille millisecondes, le coût est de l'ordre d'un à cinq centimes par appel selon le LLM choisi. Cinq nano-agents Mode C sont fournis : summarize_strict pour la summarization sous contrainte de sentence count, extract_structured pour l'extraction structurée via LLM, classify_topic_llm pour la classification de topics, sentiment_nuanced pour le sentiment plus subtil que les classifiers locaux, et qa_grounded pour le question-answering avec citation des sources.

28. Le catalogue et les tiers de stabilité

Chaque nano-agent dans le système EFE est décrit par un manifest typé et versionné qui inclut son identifiant, sa version semver, sa catégorie (extraction, validation, classification, transformation, lookup, entity recognition, summarization, structured extract, ou other), son auteur (EFE shipping, customer publisher, community), son tier de stabilité, son schéma JSON d'entrée, son schéma JSON de sortie, son schema d'erreur typé, son execution mode (A, B, ou C avec leurs paramètres respectifs), son niveau de déterminisme (full deterministic, seeded, llm replayable, llm ephemeral, ou side effect), sa politique d'idempotence (auto, manual avec TTL, ou disabled), ses limites de ressources, son coût estimé par appel, son SLA de latence p99, sa configuration MCP tool, sa politique d'audit, et ses tags de compliance.

Les tiers de stabilité structurent la promotion progressive d'un nano-agent. Le tier Experimental n'offre aucune garantie : le nano-agent peut être modifié ou retiré à tout moment, son schema peut breaker, sa sémantique peut changer. C'est approprié pour les beta releases et les expérimentations. Le tier 1 offre un schema freeze de douze mois : le contrat d'entrée/sortie ne peut pas changer pendant un an. C'est approprié pour la production standard. Le tier 2 offre un schema freeze de trente-six mois : approprié pour les usages enterprise critiques. Le tier Audit, enfin, offre un schema freeze de plus de sept ans plus un audit cosign cryptographiquement signé : c'est le tier requis pour les contextes fortement régulés comme FDA, FINMA, ou GxP, où un nano-agent doit pouvoir être re-joué sept ans plus tard avec garantie de comportement.

29. Le swarm dispatcher

Le swarm dispatcher est le moteur qui orchestre l'exécution parallèle d'un batch de nano-agents. Le client soumet une SwarmRequest qui contient un identifiant de requête, un tenant id, l'identifiant de l'agent appelant (par exemple « claude-sonnet-4-5 »), une liste d'opérations qui forment un DAG via leurs dépendances mutuelles déclarées, le parallélisme maximum autorisé (une safety cap, typiquement cent), le timeout global de la requête, optionnellement une clé d'idempotence, un flag streaming, et des audit tags. Chaque opération individuelle a un identifiant choisi par le client, une référence au nano-agent à invoquer, l'input proprement dit (qui peut être une valeur littérale, une référence à la sortie d'une autre opération via JSON path, ou un merge de plusieurs sources), une liste optionnelle de dépendances explicites, un timeout par opération, et une politique de retry.

L'algorithme du dispatcher procède en plusieurs étapes. D'abord il construit le graphe de dépendances explicite à partir des relations declared. Ensuite il vérifie qu'il n'y a pas de cycle : une référence circulaire est rejetée immédiatement avec une erreur InvalidDag. Puis il effectue un tri topologique qui assigne à chaque opération une couche numérique : la couche 0 contient les opérations sans dépendances, la couche K+1 contient les opérations dont les dépendances sont toutes dans des couches inférieures ou égales à K. À l'intérieur d'une couche, les opérations sont batchées par execution mode et envoyées au pool approprié : Mode A vers le tokio thread pool, Mode B vers le model pool, Mode C vers la queue du LLM router. Le dispatcher enforce la safety cap de parallélisme : si une couche contient plus d'opérations que le maximum autorisé, elles sont

sous-batchées avec backpressure. Les couches s'exécutent séquentiellement, mais les opérations à l'intérieur d'une couche s'exécutent en parallèle.

Le dispatcher gère les failure modes avec une discipline chaîtement : si une opération timeout, elle est marquée Failed et toutes ses dépendantes sont automatiquement cancelled. Si une opération retourne une erreur typée, elle est marquée Failed et ses dépendantes sont marquées Skipped (distingueré de Cancelled). Si le quota tenant est dépassé en cours de batch, le batch est terminé et les résultats partiels sont retournés avec un statut de PartialFailure. Si le LLM provider est en panne, le router M21 fait du failover automatique vers une alternative. Si une microVM Firecracker crash, l'opération est marquée Failed et la microVM est recyclée par le pool. Si une partition réseau survient, le client reçoit les résultats partiels et un hint de reconnection.

30. Les vingt-sept sprints N11 à N27

Le Mode Nano n'a pas été construit en une seule fois. Il a été livré en vingt-sept sprints numérotés N1 à N27, chacun apportant un sous-système spécifique. Les premiers sprints N1 à N10 ont établi le scaffolding initial : les types du SDK public, le runtime in-memory de Mode A, le dispatcher de base, les premiers starter pack agents. Les sprints suivants ont enrichi la plateforme progressivement.

Le sprint N11 a apporté le pool Firecracker pour la production des microVMs Mode B et Mode C, avec un fallback stub pour les environnements de développement qui n'ont pas KVM. Le sprint N12 a renforcé les clés de cache pour prévenir les fuites cross-tenant via le cache des embeddings. Le sprint N13 a introduit le mode shadow canary pour la promotion des nano-agents : une nouvelle version peut être déployée en parallèle avec l'ancienne, avec une comparaison automatique des outputs pour valider la non-régression. Le sprint N14 a ajouté le swarm envelope tracing pour l'observabilité des swarms complèts. Le sprint N15 a implémenté la fairness scheduling avec admission gate, DRR (Deficit Round Robin), token bucket, et tracking des inflight ops par tenant pour prévenir les noisy neighbors.

Le sprint N16 a ajouté le replay multi-région, qui permet de re-jouer une exécution historique dans une région différente de celle où elle a été initialement effectuée, avec gestion des clés cryptographiques KMS et shipment des chaines d'audit signées. Le sprint N17 a livré le LLM provenance record, un objet forensique de quarante-quatre champs qui décrit chaque appel LLM en mode C : identité du model avec hash cryptographique, hash de l'input prompt, paramètres de sampling, token usage, cost accounting, finish reason, safety flags, tool calls effectués, stratégie PII appliquée, tokens de redaction utilisés, publisher ID snapshot, KYC tier snapshot, gold case ID si applicable, replay status si applicable, cache hit status si applicable, et timestamp de l'écriture du record.

Le sprint N18 a livré le système d'identité des publishers tiers : signatures Ed25519, échelle KYC sur quatre tiers, cascade de révocation, sanctions screening hookable vers un provider externe comme OFAC, et publish ledger HMAC-chaîné pour l'audit. Le sprint N19 a livré le gold test gate Tier 1 : ensembles de tests immuables approuvés par opérateurs, runner hérmétique Firecracker, trois stratégies d'assertion (strict equality, numeric tolerance, semantic similarity), drift sweep automatique toutes les vingt-quatre heures, et mécanisme de bypass d'urgence avec rationale obligatoire de cent caractères minimum.

Le sprint N20 a apporté les embeddings réels via Cohere comme provider de production, avec un fallback stub pour les environnements de dev. Le sprint N21 a livré le swarm orchestrator et l'agentic bridge qui permettent à un agent IA Claude de consommer le swarm EFE via le protocole tool_use

Anthropic. Le sprint N22 a livré la HTTP gateway, qui expose vingt et un endpoints REST en plus du gRPC, avec authentification JWT, OpenAPI 3.1, Prometheus metrics, graceful shutdown, body limits, et rate limits. Le sprint N23 a livré l'adapter NanoAgent vers SwarmAgent qui permet d'utiliser un nano-agent du catalogue M50 dans le contexte d'un swarm N21 avec toutes les garanties signature, gold, idempotence préservées.

Le sprint N24 a livré le wired compliance runner : au lieu d'avoir des gates de compliance optionnels et configurables, ce sprint les a wirés par défaut pour les nano-agents tier 1 et plus, garantissant qu'aucun nano-agent non signé ou sans gold-binding ne peut s'exécuter en production. Les sprints N26 et N27, les plus récents, ont apporté respectivement la wiring complète du LLM provenance depuis le ClaudeBridge, et le streaming Anthropic SSE end-to-end avec retry exponentiel et un wiring provenance compatible streaming. Le N27 a vu plusieurs passes d'audit chirurgicales qui ont identifié et corrigé neuf bugs subtils, tous documents dans des passes successives.

Partie VIII — La couche audit transverse

31. La chaîne HMAC par tenant

La couche audit d'EFE est transverse : elle wrappe les opérations des couches 1, 2, et 4 sans être un module isolé. Le mécanisme fondamental est la chaîne HMAC par tenant, livrée initialement avec le module M23 puis étendue par le M50.5. L'idée est simple : chaque opération significative produit une ligne dans la table audit_chain, et chaque ligne est linée cryptographiquement à la précédente par un HMAC qui inclut le hash de la précédente. Formellement, pour la ligne numéro N, le payload_hmac est calculé comme HMAC(tenant_key, prev_hmac concaté avec canonical(payload)).

Cette structure chaînée a une conséquence cryptographique très importante : il est impossible de modifier rétroactivement une ligne de la chaîne sans casser tous les HMAC subséquents. Si un attaquant tente d'altérer la ligne dix-sept pour effacer une opération gênante, le HMAC de la ligne dix-huit ne sera plus valide, parce qu'il a été calculé avec le hash de la ligne dix-sept originale. Même chose pour la ligne dix-neuf, et ainsi de suite jusqu'à la fin de la chaîne. Pour falsifier la chaîne, l'attaquant devrait recalculer tous les HMAC subséquents, ce qui exige de connaître la clé HMAC du tenant — clé qui est stockée dans un KMS et jamais exposée en clair.

Cette propriété rend la chaîne HMAC equivalent à un journal append-only cryptographiquement immuable. Un régulateur qui auditerait la plateforme peut vérifier l'intégrité de la chaîne en re-calculant les HMAC depuis le début et en comparant avec les valeurs stockées. Cette vérification est linéaire en le nombre de lignes, ce qui peut devenir coûteux pour des tenants à haut volume qui produisent des milliards de lignes par an. C'est pour résoudre ce problème de scaling qu'a été ajoutée la couche Merkle epoch.

32. Les Merkle epochs pour l'inclusion proof

Le sprint N9 du Mode Nano a introduit une couche additionnelle au-dessus de la chaîne HMAC linéaire : les Merkle epochs. L'idée est de regrouper les lignes de la chaîne en epochs de taille fixe — typiquement mille ou dix mille lignes — et de construire un arbre de Merkle sur chaque epoch. La racine de l'arbre Merkle est ensuite publiée dans un EpochRootStore qui peut être une base de données propre ou même un timestamping service externe comme une blockchain publique pour les usages les plus sensibles.

Cette structure permet des inclusion proofs en O(log N) au lieu de O(N) pour la chaîne linéaire. Pour prouver qu'une ligne spécifique était présente dans la chaîne à un moment donné, il suffit de fournir la ligne, l'epoch root correspondant, et le chemin Merkle de log N hash intermédiaires. Cela rend l'audit pratique même pour des chaînes à milliards de lignes. La signature des epoch roots est elle-même cryptographique : un EpochSealer signe chaque root avec une clé d'opérateur séparée de la clé HMAC, ajoutant une couche de défense en cas de compromission de la clé HMAC.

33. Le replay engine

L'audit chain par construction n'est utile que si on peut effectivement re-jouer les exécutions historiques. C'est le rôle du replay engine d'EFE. Le pipeline complet est : l'opérateur soumet une requête ReplayEngine.replay avec le tenant_id et le request_id de l'exécution originale. Le système fetch toutes les lignes d'audit associées à ce request_id. Il vérifie l'intégrité cryptographique des HMAC sur cette sous-chaîne. Il reconstruit une SwarmRequest fraîche à partir des traces. Il dispatch

cette requête dans le dispatcher actuel. Il compare les outputs opération par opération avec les outputs stockés dans la trace originale via le DeterminismChecker. Et il assemble un ReplayReport structuré.

Le rapport de replay distingue cinq résultats possibles par opération. Match signifie que l'output reproduit exactement l'output original : c'est le cas attendu pour les nano-agents Mode A FullDeterministic. Mismatch signifie que les outputs diffèrent : c'est un signal de régression qui doit être investigué. ReplayWindowExpired signifie qu'on a essayé de re-jouer un nano-agent Mode C en mode replayable mais la fenêtre de cache LLM est expirée : l'output original ne peut plus être obtenu déterministiquement, même en ré-appelant le même LLM avec le même prompt. SkippedNonDeterministic signifie qu'on a explicitement marqué ce nano-agent comme éphémère : on ne tente même pas de le re-jouer. Et MissingFullInput signifie que la trace ne contient pas l'input complet nécessaire au re-play, typiquement parce que la rétention de payload complet est expirée.

34. La provenance LLM par appel

Le module N17 a introduit une couche d'audit spécifique aux appels LLM : le LlmProvenanceRecord. Chaque invocation d'un nano-agent Mode C ou d'un appel direct à un LLM via le bridge produit exactement un record de quarante-quatre champs. Ces champs documentent tout ce qui est nécessaire pour prouver l'integrité d'un appel des années plus tard : l'identité du tenant qui a fait l'appel, l'identifiant du trace de la session, l'opération spécifique, l'agent appelant, l'identité complète du model (provider, model_id, version requestée, version servie effectivement par le provider, hash du content quand disponible), si le model hash a été vérifié ou pas, le template de prompt utilisé (id et version), les hash des composants du prompt (inputs, system, user, final après composition), les paramètres de sampling, le token usage (input et output), le cost accounting (détail des prix), l'identifiant de requête du provider, le temps de réponse provider, le finish reason, la région qui a servi l'appel, les métadonnées provider additionnelles, le hash du contenu de sortie après redaction PII, la completeness (Complete, Partial, Aborted, Refused, Errored), les safety flags éventuels, les tool calls effectués pendant l'appel, la stratégie PII appliquée, les tokens de redaction utilisés, l'identifiant du publisher s'il y en a un, le KYC tier au moment de l'appel, l'identifiant du gold case si applicable, le replay of si c'est un replay, le cache hit info si c'est un cache hit, le timestamp d'enregistrement, et les nouveaux champs N27 streaming_mode, retry_count, retry_outcome, et time_to_first_token_ms.

Ce record est volontairement verbeux. Il est conçu pour qu'un auditeur, des années après, puisse reconstituer complètement le contexte d'un appel particulier sans avoir à faire d'archeologie sur les logs ou la base de données. Chaque champ a une raison : le hash du model permet de détecter un silent swap où le provider aurait changé son model sans notification, le hash du prompt permet de prouver exactement ce qu'on a envoyé au LLM, le cost accounting permet de vérifier que la facturation correspond à ce qui a réellement été consommé, le KYC tier snapshot préserve l'état exact du publisher au moment de l'appel même si son tier a changé depuis.

35. Le gold test gate Tier 1

Le sprint N19 a introduit le gold test gate, un mécanisme de gating qualité pour la promotion des nano-agents en production. Le concept est emprunté aux pratiques QA industrielles : chaque nano-agent qui veut être promu de tier Experimental à tier 1 ou plus doit être lié à un GoldSet, un ensemble immuable de cas de test pré-approuvés par au moins deux opérateurs avec rationale écrit : GOLD_SET_MIN_APPROVERS vaut deux. Le GoldSet contient des paires (input,

expected_output) sous une des trois stratégies d'assertion. Pour les nano-agents Mode A purement déterministes, c'est strict equality : l'output doit être exactement le même que l'expected. Pour les nano-agents Mode B qui produisent des réels flottants, c'est numeric tolerance avec un epsilon explicite. Pour les nano-agents Mode C dont l'output est du texte, c'est semantic similarity via embedding cosine.

Une fois le nano-agent promu, son comportement est surveillé en continu par le drift sweep, un job cron qui re-tourne tous les vingt-quatre heures et re-exécute tous les gold cases sur la version courante du nano-agent. Si plus d'un certain pourcentage des cases échouent — configurable selon le tier — le nano-agent est automatiquement rétrogradé et une alerte est levée pour les opérateurs. Le drift sweep utilise un runner hérmétique Firecracker pour garantir que l'environnement d'exécution est identique à l'environnement de production. En cas d'urgence vérifiable — bug critique, incident production, nécessité de bypass temporaire — les opérateurs peuvent émettre un EmergencyBypass cryptographiquement signé avec rationale obligatoire de cent caractères minimum et lifetime maximum de sept jours.

36. L'identité des publishers tiers

Le sprint N18 a apporté l'infrastructure d'identité des publishers tiers, qui permet à des éditeurs externes d'EFE de publier leurs propres nano-agents avec garanties cryptographiques de provenance. Chaque publisher candidate soumet un NewPublisherCandidate qui contient son canonical body (ses métadonnées structurées), sa signature Ed25519 sur ce canonical body, et les pièces justificatives nécessaires au KYC. Le système vérifie la signature contre la clé publique déclarée, soumet les pièces au KYC provider (StubKycProvider en dev, real KYC en prod), et fait du sanctions screening contre les listes OFAC et EU sanctions.

Si tous les checks passent, le publisher est admis en tier 0 avec son ProvisionalPublisherId. Pour être promu à tier 1, il doit avoir déjà publié au moins un nano-agent qui a accumulé quatre-vingt-dix jours de production sans incident. Pour tier 2, il doit avoir une cooldown enforcement vérifiée : un délai minimum entre les tentatives de promotion qui prévient les manipulations rapides du tier. Pour tier Audit, il doit avoir un audit cosign-signed externe et un track record supérieur à deux ans. Toutes les opérations sur le publisher — création, rotation de clé, promotion, révocation — sont enregistrées dans un PublishLedger HMAC-chaîné séparé de la chaîne d'audit principale, pour rendre les opérations sur publishers vérifiables indépendamment.

Partie IX — Les patterns d'intégration

37. La taxonomie des cinq lettres A à E

EFE supporte un spectre de cinq patterns d'intégration, identifiés par les lettres A, B, C, D, et E dans la documentation produit. Cette taxonomie structure le go to market par maturité d'usage : chaque lettre correspond à un niveau de sophistication d'intégration et à un persona utilisateur distinct. La lettre A est la plus simple : l'usage CLI one-shot via le binaire efe-admin. L'utilisateur tape une commande, reçoit une réponse JSON sur stdout, et l'opération termine. Aucun code utilisateur n'est requis : pas d'import Python, pas de monkey-patching, juste une commande shell. Chaque invocation est un processus neuf, sans persistance entre invocations, sans cache, sans session réutilisée.

Le contrat strict de la lettre A est triple. Premièrement, aucun code utilisateur : le client de A est quelqu'un qui ne écrit pas de code, juste des commandes. Deuxièmement, aucune persistance : chaque invocation est isolée dans le temps, comme une fonction pure. Troisièmement, sortie machine-parseable par défaut : le stdout par défaut est un document JSON structuré et stable, ce qui permet de compose la commande avec jq ou tout autre outil de manipulation JSON. Cette discipline rend l'usage A particulièrement adapté aux scripts shell, aux pipelines CI/CD, et aux démonstrations one-shot.

La lettre B est le REPL persistant. Un utilisateur lance efe-admin sans arguments, obtient un prompt interactif, et peut taper des commandes successives qui partagent le même processus, le même catalog chargé en mémoire, la même connexion gRPC vers le serveur EFE. Cela permet l'exploration interactive : l'utilisateur fait une classification, observe le résultat, ajuste sa question, refait une classification, et ainsi de suite, avec une latence amortie par la persistance du process. Le REPL supporte le hot-reload des catalogs : si l'utilisateur ajoute des exemples dans le catalog via un autre canal, il peut faire reload depuis le REPL sans redémarrer le processus.

La lettre C est le script applicatif. C'est le mode où un utilisateur écrit du code Python qui orchestre EFE et un LLM hôte pour produire une application. L'utilisateur de C importe le SDK Python d'EFE, instancie un client, et compose ses propres workflows en alternant des appels EFE et des appels à Claude ou GPT. C imposera nécessairement du Python et du code utilisateur : ce n'est pas un usage « pour qui ne code pas ». Mais c'est le premier niveau où le client peut vraiment construire une application sur EFE plutôt que simplement consommer ses outputs.

La lettre D est l'intégration library. Ici, EFE est livré comme une bibliothèque in-process qui partage le même processus que l'application cliente. C'est particulièrement adapté aux clients qui ont des contraintes de latence strictes ou des exigences de confidentialité qui interdisent l'appel réseau : avec D, l'application embarque directement EFE et l'appelle via FFI Python ou via une interface Rust native. La latence est minimale (pas de network round-trip) et aucune donnée ne quitte le process. Le cycle de vie est couplé : si l'application cliente crash, EFE aussi ; si EFE crash, l'application cliente subit.

La lettre E est le service backend microservice. EFE tourne comme un service indépendant, exposé via gRPC et REST, et les applications clientes en multiple langages (Python, Go, Java, JavaScript, Swift, Rust) consomment ce service via leurs SDKs respectifs. C'est le pattern le plus matur de déploiement : il permet le scale-out, l'isolation des défaillances, le rolling update, et le multi-tenant strict. C'est la lettre ciblée pour les déploiements enterprise à fort volume.

38. L'écosystème MCP

En parallèle des cinq lettres, EFE expose une surface MCP — Model Context Protocol — qui est le standard adopté par Anthropic et de plus en plus par l'écosystème agent IA. Le serveur MCP d'EFE, identifié M16 dans la nomenclature des modules, expose actuellement soixante-dix-huit tools agent-callable. Ces tools couvrent neuf domaines fonctionnels : la classification de base, la curation de catalog, les opérations sur bundles, le déploiement, le SDK BYO-LLM, le LLM router, le mode 3 multi-turn, le runtime d'agent, et l'administration de tenants.

Quand un client Claude Desktop, Cursor, Zed, Continue, ou Cline déclare le serveur MCP EFE dans sa configuration, il acquiert accès à ces soixante-dix-huit tools automatiquement. Le LLM hôte — Claude ou GPT — peut alors les invoquer dans ses conversations utilisateur sans que l'utilisateur ait à coder quoi que ce soit. Cela rend EFE immédiatement utile pour des cas d'usage exploratoires : un consultant qui veut classifier rapidement un batch de feedbacks utilisateur ouvre Claude Desktop, configure EFE comme MCP server, et demande directement à Claude de classifier les feedbacks via le tool efe_classify. Aucun setup développeur n'est requis.

Le serveur MCP applique le pattern confirm-required pour les tools qui font des écritures : création de catalog, évolution, suppression, modification de quotas. Pour ces tools, l'agent IA doit fournir explicitement un argument confirm: true dans l'invocation. Cela prévient les destructions accidentelles et les attaques prompt injection où un utilisateur malveillant tenterait de faire exécuter une opération destructive par l'agent. Ce pattern de protection contre STRIDE TM-M16-T-01 est systématique sur tous les write tools.

Partie X — L'écosystème opérationnel

39. Les sprints D pour l'infrastructure opérationnelle

En parallèle des sprints M qui développent les modules fonctionnels, les sprints D adressent les besoins opérationnels nécessaires au déploiement en production à grande échelle. Le sprint D8, identifié Gloss A dans la nomenclature interne, a livré le binaire efe-admin : un CLI Rust statiquement compilé qui consomme les tools MCP m23 du serveur MCP pour permettre aux clients enterprise de gérer leurs tenants en self-service. L'opérateur enterprise peut ainsi créer des tenants, suspendre des tenants, ajuster des quotas, sans devoir passer par l'équipe EFE pour chaque opération administrative.

Le sprint D17 a livré efe-backup, l'orchestrateur de backup et disaster recovery. Tout déploiement EFE doit avoir une stratégie de sauvegarde et de restauration qui respecte les contraintes contractuelles avec le client. Efe-backup automatise les snapshots périodiques des composants critiques (la base de données relationnelle, les blobs des catalogs et bundles, les configurations Helm, les secrets KMS) vers un object store off-site avec encryption at rest. L'orchestrateur supporte trois modes : incremental, qui ne sauvegarde que les changements depuis le snapshot précédent (rapide, peu coûteux) ; full, qui sauvegarde tout (lent, plus coûteux mais autoporteur) ; et synthetic full, qui applique les incrementals sur un full pour reconstruire un nouveau full sans re-lire toute la base.

Le sprint D18 a livré efe-bench, le bench harness pour les opérations de catalog. Mesurer la performance d'opérations comme la classification à haut volume sur des catalogs réels est essentiel pour valider les SLO contractuels. Efe-bench fournit l'infrastructure pour exécuter ces benchmarks de façon reproductible : même dataset d'inputs, mêmes catalogs, même hardware, même configuration réseau. Les résultats sont comparés aux baselines historiques stockés dans bench_baselines, et toute régression significative déclenche une alerte automatique.

Le sprint D93, identifié Sprint Phi-D-LATACT dans la nomenclature, a livré efe-saga, l'orchestrateur de saga. Une saga est un pattern de gestion de transactions distribuées : quand une opération utilisateur doit modifier plusieurs sous-systèmes (la base de données, l'object store, le cache, le publisher registry, le billing), il faut une mécanique qui garantisse la cohérence globale même en cas de panne partielle. Efe-saga implémente ce pattern sous forme d'une machine à états persistente qui suit chaque saga par sa sequence d'opérations et leurs compensations associées. Si une opération échoue, la saga déclenche les compensations des opérations précédemment réussies, ramenant le système à un état cohérent.

Les cas d'usage typiques d'efe-saga incluent le self-service tenant deploy — créer un tenant implique de modifier plusieurs systèmes simultanément et doit être transactionnel — et la GDPR delete cascade. Ce dernier est particulièrement intéressant : quand un utilisateur européen exerce son droit à l'effacement, EFE doit effacer ses données de tous les sous-systèmes — audit chain, cache, vector index, knowledge graph, blobs, etc. — de façon atomique pour que l'utilisateur reçoive un certificat d'effacement valide. Efe-saga orchestre cette cascade et gère les compensations en cas d'échec partiel.

40. Le stack de benchmarking et d'observabilité

EFE est instrumenté de façon extensive parce que les contrats SLA avec les clients enterprise exigent une mesurabilité rigoureuse. Les métriques exposées couvrent quatre familles. Les métriques de latence : histogrammes Prometheus pour la latence end-to-end de chaque endpoint, decomposable en latence per-layer (parsing, classification, decoding, audit write). Les métriques d'erreur : counters par type d'erreur, par tenant, par catalog, avec corrélation possible avec les traces détaillées. Les métriques de throughput : nombre de requêtes par seconde, par tenant, par mode. Les métriques de saturation : utilisation CPU, mémoire, file descriptors, pool de connexions gRPC, taille des queues internes.

Le sous-système slos/rollup ajoute par-dessus une couche d'aggregation des SLOs. Quatre SLOs canoniques sont définis pour EFE : la disponibilité (pourcentage de minutes par mois où le service est UP), la latence p99 (95-ième percentile de la distribution des temps de réponse), le taux d'erreur (pourcentage d'appels qui échouent avec 5xx), et la durée moyenne avant restauration après incident (MTTR). Ces SLOs sont calculés à plusieurs granularités temporelles (heure, jour, semaine, mois) et à plusieurs granularités organisationnelles (par tenant, par région, global), avec rollup hiérarchique.

Le sous-système smoke/runner ajoute les smoke checks exécutés à chaque merge sur la branche principale comme gate de qualité. Neuf smoke checks couvrent les operations critiques : bootup et healthz du serveur, ping gRPC basique, classification minimale, register et evolve catalog, lecture audit log, verification des metrics endpoint, validation du health check Kubernetes, et un tear-down propre. Tous les neuf doivent passer en moins de cinq minutes pour qu'un PR puisse être mergé.

41. Le cluster provisioner et les chaos drills

Le sprint PR 23 du Meta-Swarm Phase 2 a extrait du bench harness un sous-système réutilisable : efe-cluster-provisioner. Ce crate est responsable de la création à la demande de clusters EFE éphémères pour les besoins de testing, de bench, et de chaos engineering. Le provisioner supporte plusieurs backends : kind pour le local dev, k3s pour les tests CI légers, et EKS, GKE, AKS pour les tests à grande échelle dans le cloud. La création d'un cluster suit un manifest déclaratif qui spécifie la version EFE, le nombre de répliques par service, les ressources allouées, et la configuration réseau.

Les chaos drills utilisent le provisioner pour créer des clusters test, leur injecter des fautes contrôlées via chaos-mesh, et vérifier que le système se comporte conformément aux SLA. Les fautes typiques injectées incluent : kill d'un pod aléatoire pour tester le self-healing, partition réseau entre deux zones pour tester le multi-AZ, augmentation artificielle de la latence vers la base de données pour tester les timeouts, saturation CPU d'un pod pour tester l'admission control, et corruption de bytes aléatoires pour tester la détection d'erreur. Le système doit, dans chaque scénario, soit continuer à servir correctement, soit échouer proprement avec un message d'erreur approprié et un log détaillé.

Partie XI — Le verdict honnête sur l'état réel

42. Ce qui est shippé et vérifié

Après avoir parcouru l'intégralité de l'architecture, il convient d'être rigoureux sur ce qui est réellement livré aujourd'hui dans le repository et ce qui relève encore de l'aspirationnel. Cette honnêteté est essentielle pour ne pas donner une vision déformée de la plateforme.

Du côté shippé et vérifié par les tests automatisés qui passent dans le workspace : l'intégralité du SDK Mode Nano avec ses trois cent vingt-sept tests unitaires, le runtime du Mode Nano avec ses mille cent cinquante tests, les deux cent six tests des starter pack agents (Mode A et Mode B et Mode C), le blackboard et le swarm orchestrator du N21, les six agents pre-analysis (NER, sentiment, language, schema, PII, summary), l'identité des publishers Ed25519 avec son ledger HMAC, le gold test gate Tier 1 avec ses runners et drift sweep, la LLM provenance N17/N26/N27 complète avec streaming et retry, le multi-region replay N16, la wired compliance N24, l'agentic bridge avec streaming SSE Anthropic, le HTTP gateway N22 avec ses vingt et un endpoints et l'OpenAPI 3.1.

Plus largement : les trois cent vingt-sept tests du efe-core qui couvrent les modules M1 à M9 du moteur d'inférence latente, les neuf cents tests de efe-server qui couvrent l'API gRPC M10, les trois cent trente-deux tests de efe-telemetry, le LLM router M21 avec ses provider abstractions, l'efe-prompts versionné et A/B testable, l'efe-curation pour la taxonomy curation M23, l'efe-active-learning pour le closed loop M24, l'efe-agent-runner pour le Firecracker sandbox M25, l'efe-cache pour le Redis cache plane M27, l'efe-events pour NATS pub/sub M28, l'efe-pii pour la détection et redaction M29, l'efe-multimodal pour vision audio et chunking M30, l'efe-region pour la souveraineté M30, l'efe-metering et efe-billing pour M31, l'efe-vector et efe-retrieval pour M32, l'efe-kg et efe-graphrag pour M33.

Plus l'A1 : l'efe-meta-swarm Phase 1 avec ses deux cent onze tests, l'efe-meta-swarm-proto avec ses descripteurs gRPC, l'efe-meta-swarm-server avec ses quatre cent cinquante-deux tests pour les services controller et participant. Plus le tooling : l'efe-admin CLI, l'efe-saga, l'efe-backup, l'efe-bench, l'efe-cluster-provisioner, le bench harness, les slos rollup, le smoke runner, le loadgen L2. Plus le MCP server avec ses cent quatre-vingt-sept fichiers Python qui exposent les soixante-dix-huit tools. Plus l'infrastructure : deux cent trente-deux fichiers Helm charts, soixante-seize fichiers Terraform, les SDKs Python, Go, et le contract-runner multi-langage.

Au total, le workspace contient quarante-trois crates Rust en plus de quatre sous-workspaces, mille deux cent huit fichiers Rust, plus de mille cinq cents fichiers source totalement, et un cumul d'environ sept mille cinq cents annotations de test. Les tests qui peuvent être exécutés sans contrainte d'environnement produisent plus de cinq mille tests passants vérifiés, zéro échec.

43. Ce qui est stubbé ou différé

Du côté stubbé ou différé pour des raisons d'infrastructure ou de calendrier : le pool Firecracker production reste en mode StubVmPool pour le développement ; la production avec KVM réel demande une infrastructure dedicated qui n'est pas encore déployée. Cela signifie que les microVMs Mode B et Mode C tournent actuellement in-process et non dans de vrais VMs, ce qui change le profil de sécurité mais pas la fonctionnalité. La majorité des stores sont des implementations InMemory : InMemoryCatalog, InMemoryLlmProvenanceStore, InMemoryPublisherRegistry,

InMemoryGoldSetRegistry, InMemoryHmacChainStore. Les implementations Postgres sont planifiées mais pas encore déployées : l'unique fichier SQL de migrations dans le repo confirme cet état.

Le billing Stripe est shippé en code mais le pipeline Stripe-real avec invoicing automatique et reconciliation comptable n'est pas wiré en production. Le clickhouse pour l'aggregation high-volume des métriques de métering est mentionné dans l'architecture mais n'a pas de code visible dans le repo. Les providers KYC et sanctions sont des stubs permissifs en développement : StubKycProvider::permissive et StubSanctionsScreener::clear. Les real providers (Onfido, OFAC API, Refinitiv) doivent être wirés par l'opérateur de production. La pipeline cosign signing complète pour TierAudit existe en crypto code mais le déploiement infrastructural n'est pas finalizé.

Le LLM router M21 utilise MockLlmClient et BuiltinLlmClient pour les tests : il y a aussi un real HTTP client via reqwest qui parle directement à Anthropic et OpenAI, mais le router complet avec rule-based provider selection, A/B testing, et circuit breaker sophistiqué existe partiellement. Les kernels CUDA du M5 fusion sont annoncés en Phase 3 du roadmap : le module efe-kernels existe avec le scaffolding mais l'optimisation GPU complète est en travail. Le build CUDA n'est pas dans le hot path test : le repo passe ses tests sans CUDA, ce qui suggère que la production actuelle tourne sur CPU.

44. Le verdict architectural et commercial

EFE est une plateforme technique ambitieuse, structurée en quatre couches indépendantes qui se composent verticalement. La couche 1, le moteur d'inférence latente, est l'IP différentielle : peu d'acteurs au monde construisent des moteurs multi-agents par fusion tensorielle dans un espace latent partagé. C'est cet IP qui justifie une valorisation industrielle : si EFE n'était que des appels Claude wrappés, sa moat serait nulle. La couche 2, la fédération meta-swarm, étend ce moteur à plusieurs instances coordonnées, ce qui ouvre la voie aux déploiements hybrides à grande échelle : un client peut avoir une instance EFE par département métier, par région géographique, ou par sensibilité de données, et utiliser le meta-swarm pour faire converger ces instances quand nécessaire.

La couche 3, l'audit transverse, est la couche qui rend EFE compatible avec les contextes régulés. Sans cette couche, EFE serait juste un autre moteur ML ; avec elle, EFE devient un produit pour FINMA, FDA, DORA, AI Act compliance. La couche 4, le Mode Nano, est la couche commerciale la plus récente et la plus ambitieuse : elle étend le marché adressable d'EFE en passant du B2B traditionnel au B2A. Les vingt-sept sprints N1 à N27 qui composent cette couche — et qui ont mobilisé récemment des efforts d'audit chirurgicaux comme ceux qu'on a vus dans les passes successives N27 — témoignent d'une discipline d'ingénierie très sérieuse.

Le marché ciblé est clair : secteurs régulés (pharma FDA, finance FINMA et DORA, légal, gouvernemental) où le simple « ça marche bien » ne suffit pas, où il faut prouver que ça a marché correctement à un régulateur sept ans plus tard. La différentiation commerciale est l'audit cryptographique end-to-end couvrant chaque appel, chaque round d'inférence, chaque modification de taxonomie, chaque promotion de model, chaque opération administrative. C'est une promesse forte et difficile à imiter parce qu'elle exige une discipline architecturale présente dès le début de la conception : ajouter de l'audit cryptographique rétroactivement à un produit existant est presque impossible.

Le risque commercial principal est temporel : EFE doit atteindre le go-to-market avant que les grands fournisseurs LLM (Anthropic, OpenAI) ne décident d'intégrer verticalement les capacités d'audit dans leurs propres plateformes. Pour l'instant, les fournisseurs LLM se concentrent sur la capacité LLM elle-même et laissent le compliance à des couches tierces. Si cela change, EFE doit avoir déjà capturé suffisamment de clients enterprise pour avoir une position défendable, soit comme acquéreur potentiel, soit comme jeu indépendant viable.

Le risque technique principal est l'écart entre le code shipped et l'infrastructure de production. Beaucoup de pièces architecturales sont présentes en code mais ne tournent pas encore avec leurs vraies dépendances production : Postgres réel, Firecracker réel, Stripe réel, KMS réel, cosign réel. Le travail de wiring infrastructural est important pour transformer le code shipped en service production-grade. Cela dit, le code lui-même est solide : les tests passent, le clippy strict passe, les invariants architecturaux sont enforced par tests dediés, et les sprints récents montrent une discipline d'audit chirurgicale (les neuf bugs trouvés dans les passes successives N27, tous corrigés).

En résumé : EFE est une plateforme massive (sept mille tests annotés, plus de cent vingt-quatre documents architecture, quarante-trois crates Rust, des SDKs en trois langages, soixante-dix-huit tools MCP), techniquement solide (les tests passent, le clippy passe, les invariants tiennent), commercialement différenciée (audit cryptographique end-to-end pour secteurs régulés), mais qui exige encore du travail substantiel sur le wiring infrastructural de production pour passer de « code complet et testé » à « production-grade chez un client enterprise FINMA-régulé ». Cette transition est le chantier opérationnel des prochains trimestres. Sur le plan strictement architectural et de l'ingénierie, ce qui est livré est impressionnant en volume, en cohérence, et en niveau de discipline.